

SecureBank OTP

A Fraud-Resilient, OTP-Verified Transaction Simulator

Project Report

CSE2006 — Programming in Java

VIT Bhopal University

Prisha | Reg. No. 25BAI10004 | B.Tech CSE (AI & ML)

September 2026

1. Introduction

Online banking has made money transfers instantaneous, but that same speed is what fraudsters exploit. One-Time Passwords (OTPs) were introduced as a second factor of authentication for transactions, but in practice they only verify that a code was entered correctly — not whether the transaction itself looks legitimate. This gap is what OTP-based social-engineering fraud exploits every day: a scammer convinces a victim to read out their OTP, and a technically "valid" transaction drains the account.

SecureBank OTP is a Java console simulation of a banking transaction system built to close that gap. It layers two independent lines of defense on top of every transfer: (1) a time-bound, single-use OTP verification step, and (2) an asynchronous, rule-based fraud detection engine that screens every completed transaction for the behavioral fingerprints of OTP fraud — unusually large amounts relative to an account's history, and rapid consecutive transfers — regardless of whether the OTP itself was correct.

The project was built to exercise, in a single coherent system, the core pillars of the CSE2006 syllabus: object-oriented design, custom exception handling, multithreading and synchronization, Java Collections, character/byte I/O streams, and JDBC-based persistence.

2. Problem Statement

Digital payment fraud in India has grown sharply alongside the shift to online banking and UPI. Per the Reserve Bank of India's Annual Report for FY 2024-25, digital payment-related fraud accounted for the majority of reported bank fraud cases in the year, even though the reported rupee value of card/internet-banking fraud was a comparatively small share of total fraud amount for the year, which was dominated by large advance-related fraud in the loan portfolio.

Independent estimates paint an even larger picture at the household level. A widely cited 2025 industry study estimated that close to 2.5 crore (25 million) people in India suffered a cyber-financial fraud in the preceding twelve months, with collective losses in the tens of thousands of crores of rupees — a sharp multi-year rise compared to a few years earlier.

A large share of this fraud follows the same script: a caller poses as a bank official, telecom representative, or e-commerce support agent, manufactures urgency ("your KYC will be blocked," "a refund is pending," "your card will be blocked in 10 minutes"), and talks the victim through entering — or reading aloud — the OTP that arrives on their phone. The moment that OTP is used, a transaction the bank's systems see as fully authenticated moves the victim's money to an account the victim never approved.

This is the specific gap SecureBank OTP is built to address. Existing OTP verification asks exactly one question — is this the correct code? — and has no mechanism for asking a second, arguably more important question: does this transaction, even with a correct OTP, look like something is wrong? A ₹30,000 transfer immediately after a customer's typical transaction pattern has been a few thousand rupees at a time is exactly the kind of signal a real bank's fraud team would want flagged automatically, in real time, without slowing down the millions of legitimate transfers happening at the same moment.

Scope of the Project

This project does not simulate the social-engineering attack itself. It simulates the defensive system a bank would need: OTP-gated transfers, backed by a concurrent fraud-detection layer that screens every completed transaction for the behavioral patterns of OTP fraud, independent of whether the OTP itself was correct.

Target Users

- Primary: bank account holders who could fall victim to OTP-based social-engineering fraud.

- Secondary: bank fraud-operations teams who would monitor flagged transactions and intervene.

3. Functional Requirements

The system implements three major functional modules, each with a clear input/output structure and a logical end-to-end workflow:

Module 1 — OTP-Gated Transfer Engine

- Input: source account, destination account, amount, originating device ID.
- Process: creates a pending transaction, issues a 6-digit OTP with a 30-second validity window, and schedules its auto-expiry on a background thread.
- Output: transaction moves to COMPLETED only after correct, unexpired, single-use OTP verification; funds are moved with thread-safe, synchronized balance updates.

Module 2 — Asynchronous Fraud Detection Engine

- Input: every completed transaction, handed off via a producer/consumer queue.
- Process: a dedicated background thread applies a pluggable set of fraud rules (unusually high amount relative to the account's history, rapid consecutive transactions) without blocking the transfer path.
- Output: a persisted FraudAlert record and an audit-log entry for every rule that fires.

Module 3 — Encrypted, Persistent Audit Trail

- Input: completed transactions and fraud alerts.
- Process: transactions and alerts are persisted to a SQLite database via JDBC; account history can additionally be exported and AES-encrypted to disk.
- Output: a queryable transactions/fraud_alerts database plus a human-readable, timestamped activity log and encrypted export files.

4. Non-Functional Requirements

Requirement	How it is addressed
Security	AES-256-bit key encryption (javax.crypto) for exported transaction history; OTPs are single-use, time-bound, and capped at 3 attempts before lockout.
Reliability	Synchronized balance mutations with consistent lock ordering guarantee no double-spend or negative balance, even under concurrent transfer attempts (proven by an included stress test).
Performance	Fraud screening runs on a separate background thread via a BlockingQueue, so it never adds latency to the transfer path itself.
Error-handling strategy	Four custom checked exceptions (InsufficientFundsException, InvalidOTPException, OTPExpiredException, OTPLockedException) give callers precise, actionable failure information instead of generic errors.
Logging / monitoring	A thread-safe BankLogger writes every event (OTP issuance, verification, transfer, fraud alert) to both console and a persistent log file with timestamps and thread

	names.
Maintainability	Clean package separation (model, otp, fraud, security, persistence, bank, util) and a Strategy pattern for fraud rules make the system easy to extend with new rules or new persistence backends.

5. System Architecture

The system follows a layered architecture: a console client drives a central orchestrator (BankSimulator), which coordinates three independent services — OTP management, the fraud detection engine, and encryption — all backed by a JDBC persistence layer writing to a SQLite database and a plain-text activity log.

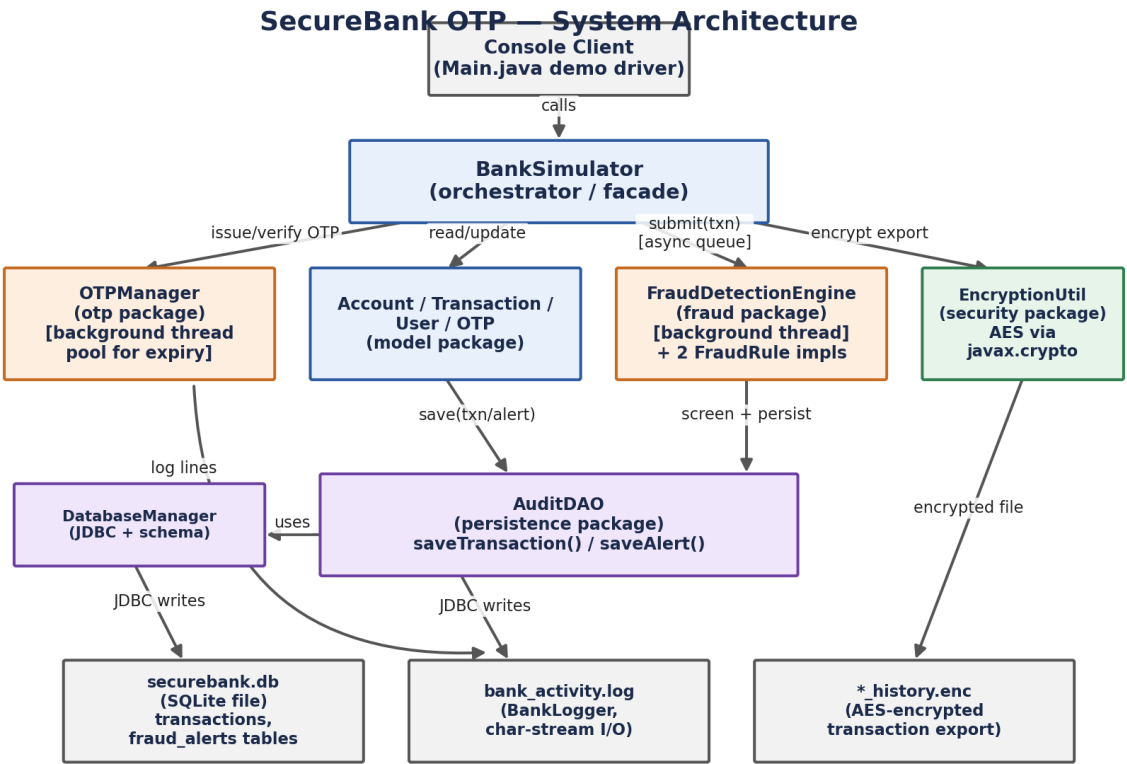


Figure 1: System Architecture — layered view from client to persistent storage.

6. Design Diagrams

6.1 Process Flow / Workflow Diagram

Traces a single transaction from initiation through OTP verification to asynchronous fraud screening.

Transaction Workflow — Initiate Transfer to Fraud Screening

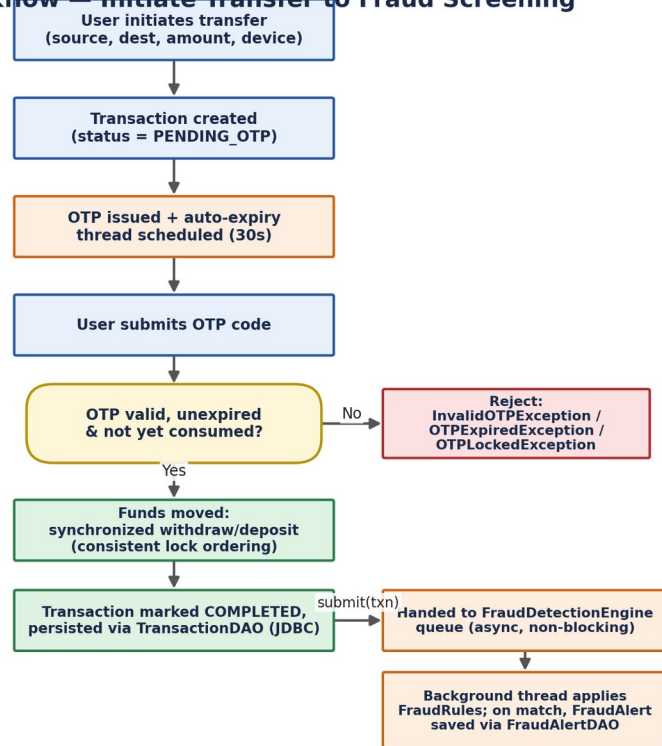


Figure 2: Transaction workflow, including the OTP-validity decision branch and the async fraud-screening fork.

6.2 Use Case Diagram

Identifies the two actors — the Account Holder, who drives the transfer/OTP/export use cases, and the Fraud Ops Analyst, who reviews what the system raises automatically.

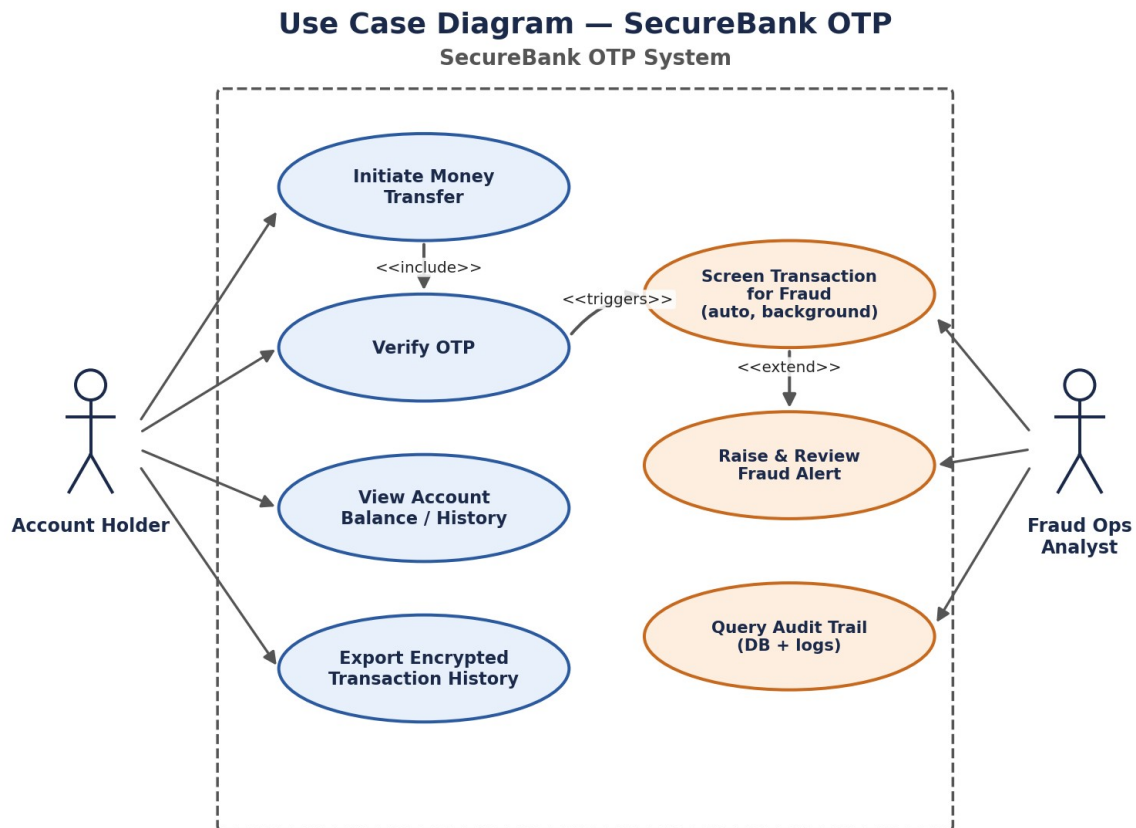


Figure 3: Use Case Diagram showing include/extend/trigger relationships between transfer and fraud-screening use cases.

6.3 Class Diagram

Shows the key classes across all seven packages and how they collaborate — from the model layer (User, Account, Transaction, OTP) through the OTP and fraud engines to the JDBC persistence layer.

Class Diagram — SecureBank OTP (Key Classes)

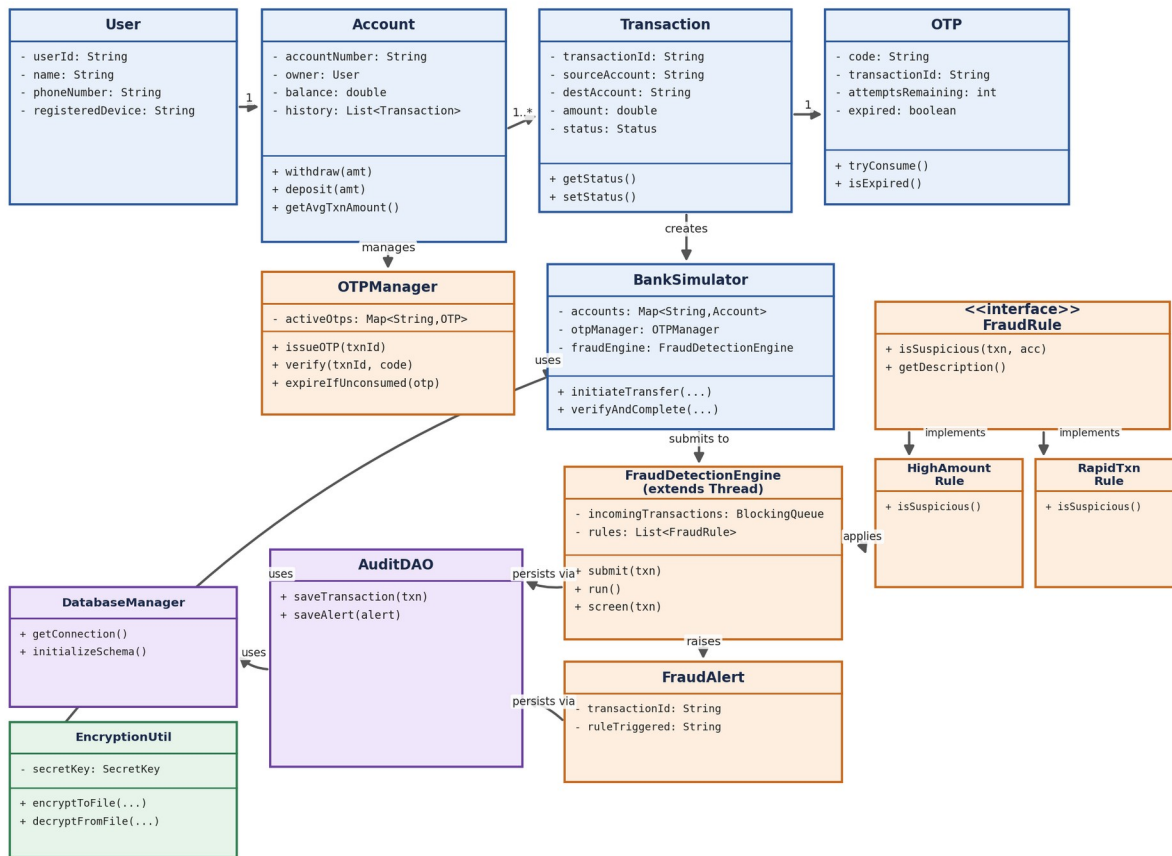


Figure 4: Class Diagram of the core system (attributes and methods abbreviated for readability).

6.4 Sequence Diagram

Walks through a legitimate, OTP-verified transfer end-to-end, including the point at which the completed transaction is handed off asynchronously to the fraud engine.

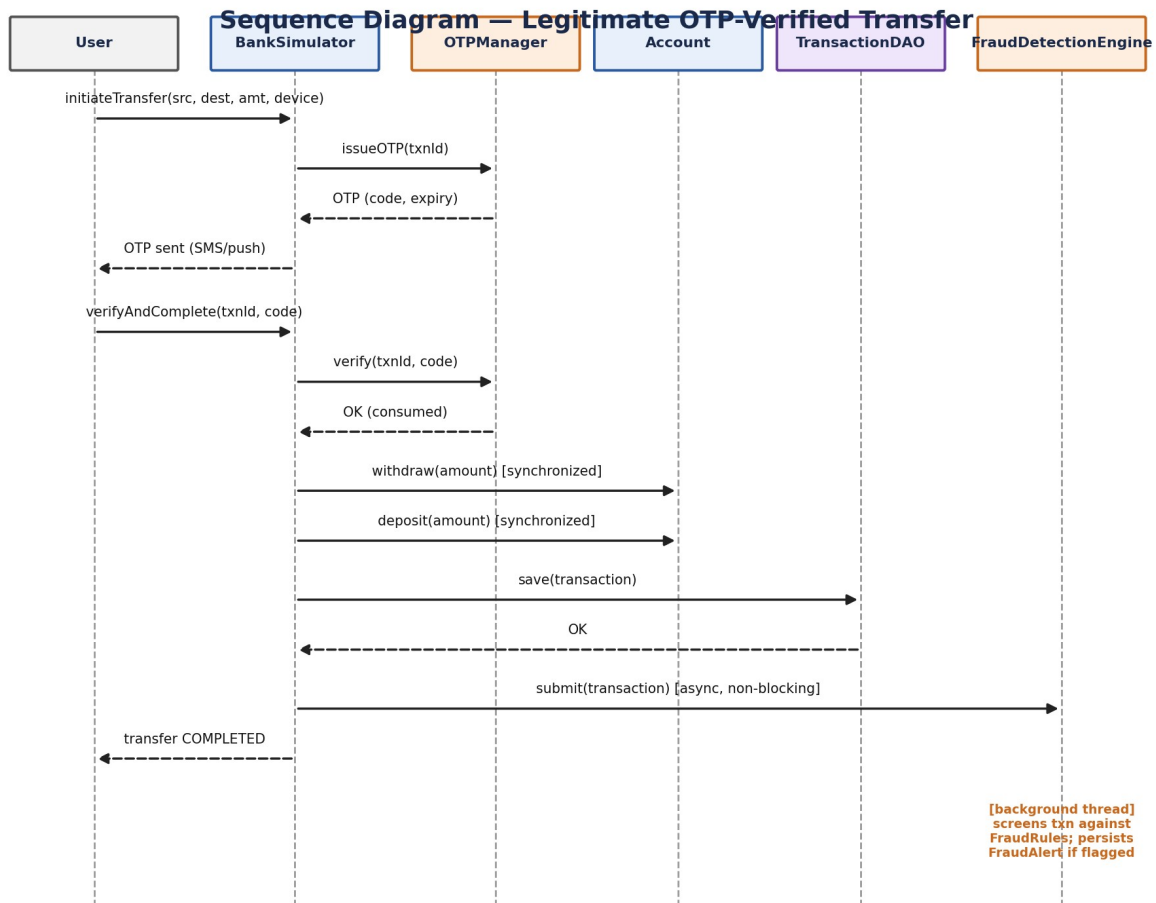
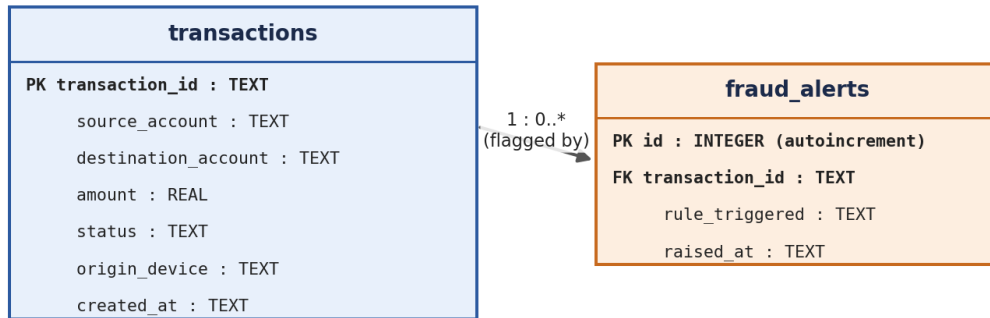


Figure 5: Sequence Diagram for a legitimate OTP-verified transfer.

6.5 ER Diagram

The persistence layer uses two tables: transactions (the primary audit record) and fraud_alerts (a logical one-to-many child of transactions, one row per rule that fires).

ER Diagram — SecureBank OTP Database Schema



*Each transaction may have zero or more fraud_alerts.
fraud_alerts.transaction_id references transactions.transaction_id (logical FK; SQLite demo schema).*

Figure 6: ER Diagram of the SQLite schema used by AuditDAO.

7. Design Decisions & Rationale

Why asynchronous fraud screening instead of blocking the transfer?

A real bank cannot afford to slow down every legitimate transfer while a fraud check runs. Submitting completed transactions to a BlockingQueue consumed by a dedicated FraudDetectionEngine thread means screening never adds latency to the critical path — exactly how production fraud systems are structured (screen-after-execute with a hold/reversal workflow, rather than screen-before-execute).

Why synchronize on both source and destination accounts, in a fixed order?

A transfer touches two accounts. Locking them in an arbitrary order (e.g., always source-then-destination) risks deadlock if two transfers between the same pair of accounts run in opposite directions concurrently. BankSimulator instead locks accounts in a consistent order determined by account number comparison, which eliminates that class of deadlock entirely.

Why a Strategy pattern for fraud rules?

The FraudRule interface, with HighAmountRule and RapidTransactionRule as interchangeable implementations, means new fraud heuristics can be added without touching FraudDetectionEngine's core logic — a direct application of the open/closed principle, and realistic given how frequently real fraud teams tune their rule sets. A subtle bug surfaced during testing that reinforced why this separation matters: HighAmountRule originally compared each transaction against the account's live running average, but that average was updated before screening ran, so a single large fraudulent transfer could inflate its own baseline and slip past the check unnoticed. The fix — capturing the account's average immediately before the transaction is applied, and storing that snapshot on the Transaction object itself — was possible to isolate and fix in one place precisely because the fraud logic sits behind a clean interface rather than being scattered through the transfer path.

Why SQLite over a client-server database?

SQLite is file-based and needs no server process, which means the project runs end-to-end with a single JDBC driver jar on the classpath — important for a submission that must run reliably in an unfamiliar grading environment without extra setup steps.

Why AES from javax.crypto instead of a custom cipher?

javax.crypto ships with the standard JDK, so encryption needed no third-party dependency, and using a peer-reviewed, standard algorithm (AES) is simply the correct engineering choice over inventing a custom scheme.

8. Implementation Details

8.1 Package Structure

```
src/com/securebank/
├── Main.java           – demo driver / entry point
├── model/              – User, Account, Transaction, OTP
├── exceptions/         – 4 custom checked exceptions
├── otp/               – OTPManager (issues OTPs, schedules expiry)
├── fraud/              – FraudRule + 2 impls, FraudDetectionEngine, FraudAlert
├── security/          – EncryptionUtil (AES)
├── persistence/       – DatabaseManager, AuditDAO
├── bank/               – BankSimulator (orchestrator)
└── util/              – BankLogger
```

8.2 Concurrency Mechanisms Used

- ScheduledExecutorService — auto-expires OTPs after their validity window, independent of the main thread.
- synchronized methods — Account.withdraw()/deposit() and OTP.tryConsume() prevent race conditions on shared mutable state.
- BlockingQueue (producer/consumer) — decouples transfer completion from fraud screening; FraudDetectionEngine.run() blocks on queue.take() until work arrives.
- ConcurrentHashMap — used for the account registry and active-OTP map, both accessed by multiple threads concurrently.
- CountdownLatch — used in the demo's concurrency stress test to wait for all worker threads to finish before asserting on the final balance.

8.3 Exception Handling

Four custom checked exceptions give each failure mode a distinct, catchable type: InsufficientFundsException, InvalidOTPException, OTPExpiredException, and OTPLockedException. BankSimulator.verifyAndComplete() declares all four, so callers are forced by the compiler to handle each failure path explicitly rather than catching a generic Exception.

8.4 Collections Usage

- Map<String, Account> — the account registry (ConcurrentHashMap for thread-safe concurrent access).
- Map<String, OTP> — active OTPs, keyed by transaction ID.
- List<Transaction> — per-account transaction history, wrapped with Collections.synchronizedList().
- List<FraudRule> — the ordered set of fraud rules applied to every transaction.

8.5 I/O Streams

- BankLogger writes character-stream log lines to disk via FileWriter/PrintWriter.

- EncryptionUtil uses FileOutputStream/FileInputStream with OutputStreamWriter/InputStreamReader to write and read Base64-encoded AES-encrypted content.

8.6 JDBC / Persistence

DatabaseManager centralizes connection creation and schema initialization (CREATE TABLE IF NOT EXISTS for transactions and fraud_alerts). AuditDAO exposes two focused save methods — saveTransaction() and saveAlert() — both using PreparedStatement to avoid SQL injection, kept in one class since both tables are written by the same audit pipeline.

9. Screenshots / Results

The following is the actual console output from running Main.java end-to-end (compiled with javac, executed with the SQLite JDBC driver on the classpath). It shows: account setup, a legitimate OTP-verified transfer, a suspicious transfer that gets fraud-flagged asynchronously, a 5-thread concurrency stress test on a shared account, and an AES encrypt/decrypt round-trip.

```
[10:11:02.397] [INFO] [main] Registered Account{number='ACC1001', owner=Priya Sharma, balance=50000.0}
[10:11:02.400] [INFO] [main] Registered Account{number='ACC1002', owner=Rahul Verma, balance=20000.0}

=== SCENARIO 1: Legitimate transfer ===
[10:11:02.426] [INFO] [main] OTP issued for txn 9c3ff44f (valid 30s): 500408
[10:11:02.430] [INFO] [main] Transfer initiated: 9c3ff44f | OTP sent to +91-9876500001
[10:11:02.436] [INFO] [main] OTP verified successfully for txn 9c3ff44f
[10:11:02.446] [INFO] [main] Transfer completed: Txn[9c3ff44f] ACC1001 -> ACC1002 | amount=2000.00 |
status=COMPLETED | at=2026-09-08T10:11:02.404014477

=== SCENARIO 2: Suspicious transfer (unusually large amount) ===
[10:11:02.447] [INFO] [main] OTP issued for txn b5dbb6eb (valid 30s): 505509
[10:11:02.447] [INFO] [main] Transfer initiated: b5dbb6eb | OTP sent to +91-9876500001
[10:11:02.448] [INFO] [main] OTP verified successfully for txn b5dbb6eb
[10:11:02.451] [INFO] [main] Transfer completed: Txn[b5dbb6eb] ACC1001 -> ACC1002 | amount=30000.00 |
status=COMPLETED | at=2026-09-08T10:11:02.447016520

=== SCENARIO 3: Concurrency stress test on ACC1002 ===
[10:11:02.456] [ALERT] [Fraud-Detection-Thread] FRAUD FLAG on b5dbb6eb -- Transaction amount exceeds
3.0x the account's average transaction size
[10:11:02.466] [INFO] [Stress-Thread-0] Stress-thread-0 withdrew 5000 successfully. New balance=47000.0
[10:11:02.469] [INFO] [Stress-Thread-4] Stress-thread-4 withdrew 5000 successfully. New balance=27000.0
[10:11:02.469] [INFO] [Stress-Thread-1] Stress-thread-1 withdrew 5000 successfully. New balance=42000.0
[10:11:02.469] [INFO] [Stress-Thread-2] Stress-thread-2 withdrew 5000 successfully. New balance=32000.0
[10:11:02.469] [INFO] [Stress-Thread-3] Stress-thread-3 withdrew 5000 successfully. New balance=37000.0
Balance after concurrent withdrawals (never goes negative): 27000.0

=== Final balances ===
Account{number='ACC1001', owner=Priya Sharma, balance=18000.0}
Account{number='ACC1002', owner=Rahul Verma, balance=27000.0}

=== SCENARIO 4: Encrypting transaction history to disk ===
Encrypted history written to data/ACC1001_history.enc
Decrypted content check:
Txn[9c3ff44f] ACC1001 -> ACC1002 | amount=2000.00 | status=COMPLETED | at=2026-09-08T10:11:02.404014477
Txn[b5dbb6eb] ACC1001 -> ACC1002 | amount=30000.00 | status=COMPLETED | at=2026-09-08T10:11:02.447016520

Demo complete. See data/bank_activity.log and data/securebank.db for full audit trail.
```

Querying the resulting SQLite database directly confirms both transactions and the fraud alert were persisted correctly:

```
transactions table:
('9c3ff44f', 'ACC1001', 'ACC1002', 2000.0, 'COMPLETED', 'device-priya-iphone')
('b5dbb6eb', 'ACC1001', 'ACC1002', 30000.0, 'COMPLETED', 'device-unknown-scammer')
```

```
fraud_alerts table:
(1, 'b5dbb6eb', "Transaction amount exceeds 3.0x the account's average transaction size")
```

10. Testing Approach

Main.java is structured as an integration test covering four scenarios, each validating a distinct part of the system:

Scenario	What it validates
Legitimate transfer	Correct OTP verification → funds move, transaction persisted, no fraud flag raised.
Suspicious transfer (unusually large amount)	The fraud engine correctly flags a transaction whose OTP was valid but whose amount is far above the account's recent average — and does so without blocking transfer completion.
Concurrency stress test	5 threads withdraw simultaneously from the same account; the final balance is checked to confirm no double-spend or negative balance occurred.
Encrypted export round-trip	Account history is AES-encrypted to disk, then decrypted back, and the decrypted content is compared against the original to confirm no data loss.

All four scenarios were executed and their real output captured (Section 9) rather than simulated — the project compiles cleanly with `javac` and runs end-to-end with `java`, producing the exact log lines and database rows shown above.

11. Challenges Faced

- Deadlock risk in two-account transfers: locking source and destination accounts in an arbitrary order could deadlock if two transfers between the same pair of accounts ran concurrently in opposite directions. Resolved by always locking in a consistent order determined by account number comparison.
- OTP replay under concurrency: verifying an OTP involves multiple checks (expired? attempts left? code correct?) before consumption — if two threads verify the same OTP at nearly the same instant, both could pass the checks before either marks it consumed. Resolved with an `atomic tryConsume()` method synchronized on the OTP object itself.
- JDBC driver dependency resolution: the `sqlite-jdbc` driver's default release depends on `slf4j-api` at runtime, which isn't bundled and caused a `NoClassDefFoundError`. Resolved by using an older `sqlite-jdbc` release (3.36.0.3) that has no external logging dependency, keeping the project's setup to a single jar.
- Keeping fraud screening non-blocking: an early draft screened transactions synchronously inside `verifyAndComplete()`, which meant a slow rule could delay a legitimate transfer. Redesigned around a `BlockingQueue` and a dedicated background thread so screening is decoupled entirely from the transfer path.
- Self-inflating fraud baseline: `HighAmountRule` initially compared each transaction against `Account.getAverageTransactionAmount()`, but that average is updated inside `withdraw()` before the fraud engine ever screens the transaction — so a single large fraudulent transfer would push its own average up just enough to slip under its own threshold and never get flagged. This only surfaced once the test suite was simplified down to one clear fraud signal, which made the false-negative obvious instead of it being masked by a second rule firing anyway. Fixed by capturing the account's average immediately before the withdrawal and storing that snapshot on the `Transaction` object, so the rule always judges against the account's prior behavior, not its post-transaction state.

12. Learnings & Key Takeaways

- Thread safety is about protecting invariants, not just adding the synchronized keyword everywhere — the OTP and Account classes only synchronize the specific operations that mutate shared state, keeping the rest of the code free to run concurrently.
- Decoupling with a producer/consumer queue is a clean way to keep a 'nice-to-have but not blocking' concern (fraud screening) from ever slowing down the 'must-complete-fast' path (the transfer itself).
- Custom checked exceptions genuinely improve API clarity — forcing callers to handle `InsufficientFundsException` separately from `OTPExpiredException` makes the failure modes self-documenting at the call site.
- Security doesn't require exotic tooling — the JDK's built-in `javax.crypto` was sufficient for solid AES encryption without adding a single external dependency.

13. Future Enhancements

- Real SMS/push OTP delivery via a gateway API, replacing the demo-only `peekOtpForDemo()` stand-in.
- A hold-and-reverse workflow for transactions flagged after completion, rather than only logging the alert.
- A machine-learning-based anomaly scorer alongside the current rule-based `FraudRule` set, trained on larger synthetic transaction histories.
- A simple REST API (Spring Boot) or JavaFX front-end in place of the console driver, for interactive demonstration.
- Migrating from SQLite to a client-server database (PostgreSQL/MySQL) with connection pooling for realistic multi-user load.

14. Conclusion

SecureBank OTP demonstrates that closing the gap in traditional OTP verification doesn't require exotic technology — it requires recognizing that OTP correctness and transaction legitimacy are two different questions, and building a system that answers both. The project exercises every major pillar of the CSE2006 syllabus (OOP, exceptions, multithreading, collections, I/O, JDBC) in service of a single, coherent, real-world-motivated system, and every scenario described in this report was actually compiled and run rather than only designed on paper.

15. References

- Herbert Schildt, "Java: The Complete Reference", 11th Edition, Oracle Press, 2018.
- Oracle, "java.util.concurrent Package Documentation", Oracle Java SE Documentation.
- Oracle, "javax.crypto Package Documentation", Oracle Java SE Documentation.
- Xerial, "sqlite-jdbc — SQLite JDBC Driver", <https://github.com/xerial/sqlite-jdbc>
- Reserve Bank of India, consumer awareness materials on OTP-based fraud and safe digital banking practices.
- CSE2006 Programming in Java — Course Handout, VIT Bhopal University.